

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2

Модульное и мутационное тестирование программного продукта

по дисциплине «Тестирование и верификация программного обеспечения»

Команда «Крутые перцы»

Автор отчета: Молдаванский Р.Д., группа ИКБО-43-24

Состав команды: Молдаванский Р.Д., Белов А.М.

Преподаватель: ассистент Левандровский А.М.

Дата выполнения: 06 октября 2026 г.

Подпись студента: _____

Подпись преподавателя: _____

Москва 2026

1 Цель и организация работы

Цель работы — разработать небольшой программный модуль, проверить модульными тестами продукт другого участника и оценить качество тестов с помощью автоматических мутаций.

В этом отчете Молдаванский Р.Д. выступает разработчиком UnitConvert и тестировщиком TextStats. За разработку TextStats отвечает Белов А.М.. Все расчеты и проверки выполняются локально на Python.

Продукт	Разработчик по роли	Тестировщик по роли
UnitConvert	Молдаванский Р.Д.	Белов А.М.
TextStats	Белов А.М.	Молдаванский Р.Д.

1.1 Задачи и последовательность

Для каждого продукта подготовлены шесть функций, точные правила работы и версия 1.0 с одной намеренной ошибкой. Начальный набор тестов обнаруживает дефект. После исправления выполняются повторные тесты и измерение покрытия. Затем Cosmic Ray создает мутанты, по выжившим вариантам дополняются тесты и повторяется измерение.

Подготовлен комплект для обмена между участниками: код, документация, тесты и карточки дефектов. Результаты ниже получены фактическими автоматическими прогонами; журналы включены в комплект.

1.2 Среда и ограничения проверки

Окружение: Windows 11 Pro 10.0.26200 x64, Python 3.12.14, Tkinter 8.6; pytest 9.1.1, coverage.py 7.16.2, Cosmic Ray 8.7.0. Функции core.py не обращаются к сети, файлам или базе данных. Тесты не зависят от открытия графического окна.

Объект покрытия и мутаций — только core.py. Интерфейс app.py проверен отдельно на запуск и основные действия. Проценты в отчете не относятся ко всему приложению и не доказывают отсутствие любых возможных ошибок.

2 Собственный модуль UnitConvert

Назначение модуля — перевод температуры, длины и массы между двумя единицами измерения. Реализация состоит из core.py с чистыми вычислениями и app.py с графическим интерфейсом Tkinter. Независимый импорт core.py не открывает окно.

Шесть публичных функций и их контракты приведены в таблице.

Функция	Результат / формула	Условие
celsius_to_fahrenheit	c × (9 / 5) + 32	c ≥ −273,15
fahrenheit_to_celsius	(f − 32) × (5 / 9)	f ≥ −459,67
km_to_miles	km × 0,621371	km ≥ 0
miles_to_km	mi / 0,621371	mi ≥ 0
kg_to_pounds	kg × 2,2046226218	kg ≥ 0
pounds_to_kg	lb / 2,2046226218	lb ≥ 0

API принимает конечные int и float. Строки, bool, None и другие типы вызывают TypeError. NaN и бесконечности, недопустимые отрицательные значения и переполнение приводят к ValueError.

Граничные температуры, нулевая масса и длина допустимы. Внутренний результат не округляется; интерфейс показывает до восьми значащих цифр. Основные числовые проверки используют pytest.approx с относительной и абсолютной погрешностью 10^{-10}.

Коэффициенты длины и массы зафиксированы в ТЗ именно в указанном виде. Это исключает расхождения из-за другого округления справочных значений. Арифметика float может давать небольшую погрешность около абсолютного нуля.

Подтверждающие файлы в папке Молдаванский_РД_UnitConvert:

core.py

app.py

docs/specification.md

versions/v1_0/core.py

3 Ошибка собственного модуля и исправление

UC-001 Неверный коэффициент в переводе Фаренгейта в Цельсия

Поле	Значение
Версия обнаружения	1.0; исходный модуль сохранен в versions/v1_0/core.py
Статус	Исправлено в 1.1; автоматический повторный прогон выполнен
Серьезность	Major (серьезная): неверный результат одной функции
Шаг воспроизведения	fahrenheit_to_celsius(212)
Ожидаемый результат	100,0 °C
Фактический результат 1.0	112,5 °C

В версии 1.0 использован множитель 5 / 8 вместо 5 / 9. При 32 °F ошибка скрыта: разность равна нулю. Значения 212 °F и −40 °F выявляют неверный коэффициент.

Ошибочная строка версии 1.0:

```
return _finite_result((value - 32) * (5 / 8))
```

Исправленная строка версии 1.1:

```
return _finite_result((value - 32) * (5 / 9))
```

Дополнительный случай: −40 °F давали −45 °C вместо −40 °C. Оба отказа относятся к одной ошибочной формуле, а не к двум отдельным дефектам.

Прогон	Успешно	Отказы	Всего
Версия 1.0, исходные тесты	22	2	24
Версия 1.1, те же тесты	24	0	24

Тесты с ожидаемым правильным результатом не подгонялись под исходную ошибку. После изменения функции тот же начальный набор прошел; исходная версия оставлена для повторного воспроизведения дефекта.

Подтверждающие файлы в папке Молдаванский_РД_UnitConvert:

docs/bug_report.md

verification/v1_0_initial.log

verification/v1_1_initial.log

4 Модульные тесты чужого проекта TextStats

Разработчик тестируемого продукта — Белов А.М.; тестировщик — Молдаванский Р.Д.. Ожидаемые значения взяты из спецификации TextStats, а не из наблюдаемого поведения дефектной версии.

Применены классы эквивалентности, обычные расчетные примеры и выбранные границы. Каждая параметризованная проверка проверяет результат одной функции. Результаты функций сравниваются с ожидаемыми значениями с помощью assert и pytest.

Группа проверок	Исходные примеры
Количество символов	Обычная кириллица; пробел и перенос строки; пустая строка
Слова и предложения	Пунктуация; повторные слова; незавершённое предложение; пустой ввод
Длина и частоты	Самое длинное слово; повторные слова; регистр; пустой словарь
Палиндром	«А роза упала на лапу Азора»; отрицательный пример «кот»; пустой ввод

Начальный набор содержит 17 тестовых случаев. В исходной версии 1.0 успешных проверок — 16, отказов — 1. Оба этапа выполнялись с одинаковыми ожидаемыми значениями.

4.1 Найденный дефект чужого модуля

TS-001: разный регистр разделяет частоты одинаковых слов. Вызов word_frequency("Кот кот КОТ") должен вернуть {"кот": 3}, но версия 1.0 возвращает {"Кот": 1, "кот": 1, "КОТ": 1}.

Замечание оформлено в docs/bug_report.md. После исправления разработчиком в версии 1.1 начальный набор дал 17 успешных проверок и ноль отказов. Это базовое условие перед запуском мутаций: исправленный модуль без изменений должен проходить тесты.

Подтверждающие файлы в папке Белов_АМ_TextStats:

tests_initial/test_core_initial.py

verification/v1_0_pytest.txt

verification/initial_pytest.txt

docs/bug_report.md

5 Покрытие кода чужого модуля

Покрытие измерено coverage.py с учетом ветвей. В область измерения включен только core.py: исполняемые строки вычислений, валидации и внутренних помощников. Тесты, app.py и инструменты запуска в показатель не входят.

Метрика	Исходный набор	После дополнения
Тестовых случаев	17	30
Исполненные строки	28 / 28	28 / 28
Покрытие строк	100,00 %	100,00 %
Исполненные ветви	10 / 10	10 / 10
Покрытие ветвей	100,00 %	100,00 %

У TextStats уже исходные 17 случаев выполнили все 28 измеряемых строк и 10 ветвей. Однако это не означает, что каждая логическая альтернатива проверена подходящими данными. Одна замена сравнения прошла исходные тесты, хотя показатель покрытия был равен 100 %.

После добавления 13 случаев покрытие осталось 100 %, но способность обнаруживать выбранные мутации повысилась. Дополнительные проверки включают Unicode, разделители слов, ничью по длине и группировку знаков конца предложения.

5.1 Как трактуются числа

Покрытие строк равно доле исполненных операторов. Покрытие ветвей оценивает посещение переходов управления. В таблице эти доли приведены отдельно; смешанный процент, который инструмент также может вывести, не подменяет ни одну из них.

Графический интерфейс не включен в автоматическую метрику. Приложение В показывает интерфейс собственного продукта автора отчета. Факты проверки чужого приложения находятся в его verification/gui_smoke.json; это проверка основных действий, а не полный набор автоматических GUI-тестов.

Подтверждающие файлы в папке Белов_АМ_TextStats:

verification/coverage_initial.json

verification/coverage_improved.json

verification/improved_pytest.txt

6 Первое мутационное тестирование

Cosmic Ray 8.7.0 автоматически создавал изменения в исправленном core.py TextStats. Применены встроенные операторы инструмента без ручного отбора отдельных успешных мутантов. Каждый запуск использовал изолированную копию исходного модуля.

Перед мутациями исходные тесты прошли на неизмененном модуле. Затем для каждого мутанта повторялся набор tests_initial. Непрохождение проверок означает обнаруженный мутант; прохождение всех проверок — выживший вариант. Ошибки инфраструктуры учитываются отдельно от результатов тестов.

Результат первого цикла	**Количество**
Сгенерировано мутантов	24
Обнаружено тестами	23
Выжило	1
Сбои / непроверенные	0
Raw mutation score	95,83 %

6.1 Почему выжил один мутант

В is_palindrome (строка 64) оператор == заменен на >=. На положительном примере строка равна своему перевороту, поэтому оба оператора дают True. Исходный отрицательный пример «кот» лексикографически меньше «ток»; оба оператора дают False.

```
# Правильно
return normalized == normalized[::-1]
# Выживший мутант
return normalized >= normalized[::-1]
```

Мутант неэквивалентен. Для «яа» обратная строка — «ая»; проверка >= ошибочно возвращает True. Это конкретный пробел в выбранных данных, который не обнаруживает само покрытие строк.

Подтверждающие файлы в папке Белов_АМ_TextStats:

docs/mutation_analysis.md

verification/mutations_initial.json

7 Дополнение тестов и повторные мутации

После разбора выживших в отдельный каталог tests_improved добавлено 13 тестовых случаев. Исходный набор сохранен. Повторный цикл использует оба каталога и тот же исправленный core.py с теми же операторами мутаций.

```
def test_palindrome_lexicographic_regression():
    assert is_palindrome("яа") is False
```

Именно этот случай обнаруживает найденную подмену == на >=. Еще 12 случаев уточняют проверку контракта: цифры и знаки пунктуации, casefold, Unicode, комбинируемые знаки, равные длины слов и повторные знаки завершения предложения. Они не выдаются за двенадцать дополнительных найденных мутантов.

Показатель	До дополнения	После дополнения
Тестовых случаев	17	30
Мутантов всего	24	24
Обнаружено	23	24
Выжило	1	0
Сбои / непроверенные	0	0
Raw mutation score	95,83 %	100,00 %

Прирост составил 4,17 процентного пункта. Все 30 итоговых тестов проходят на исправленной версии. Эквивалентные мутанты из знаменателя не исключались: в измеренном наборе они не установлены, поэтому скорректированный показатель совпадает с raw score.

Полученные 100,00 % относятся только к сгенерированным 24 вариантам TextStats. Это свидетельствует об обнаружении данного набора подмен, а не об исчерпывающем доказательстве правильности программы.

Подтверждающие файлы в папке Белов_АМ_TextStats:

tests_improved/

verification/mutations_improved.json

verification/improved_pytest.txt

8 Воспроизведение и выводы

8.1 Повторный запуск

Открыть терминал в папке Белов_АМ_TextStats. Нужен установленный Python 3.12 или новее с Tkinter. Команды ниже создают отдельное окружение .venv и устанавливают зависимости из requirements-dev.txt; активация окружения не требуется.

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.venv\Scripts\python.exe tools/verify.py initial
.venv\Scripts\python.exe tools/verify.py improved
.venv\Scripts\python.exe tools/summarize.py
.venv\Scripts\python.exe app.py
```

Инструменты создают рабочие копии и журналы в verification. Они не заменяют исправленный core.py мутированным вариантом. Повторный запуск измерений обновляет журналы, поэтому результаты комплекта стоит сохранить перед собственным экспериментом.

8.2 Выводы по своей роли

В роли разработчика подготовлен UnitConvert с шестью функциями, воспроизводимой ошибкой UC-001 в исходной версии и исправлением. На том же начальном наборе число отказов снизилось с 2 до нуля.

Как тестировщик исследован TextStats: проверен дефект TS-001, повторен исправленный расчет, измерено покрытие и сопоставлены два мутационных цикла. Набор увеличен с 17 до 30 случаев, число выживших сократилось с 1 до 0.

Основной результат — тесты усилены по наблюдаемым слабым местам, а не только ради процента покрытия. Сохраненные версии, данные и журналы позволяют независимо повторить выводы.

8.3 Источники

Методичка «Практическая работа № 2. Модульное и мутационное тестирование программного продукта», разделы 2.3–2.4.

ГОСТ 19.401-78 «Единая система программной документации. Текст программы. Требования к содержанию и оформлению». ГОСТ 34.602-2020 «Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Официальные карточки стандартов Росстандарта приведены в приложении Б.

Приложение А Исходный код своего модуля

Полный core.py UnitConvert, версия 1.1. Нумерация слева дана для чтения листинга и не является частью исполняемого файла.

```
 1  """UnitConvert 1.1: six unit conversions and explicit input validation."""
 2
 3  import math
 4
 5  __version__ = "1.1"
 6  MILES_PER_KM = 0.621371
 7  POUNDS_PER_KG = 2.2046226218
 8
 9
10  def _number(value):
11      """Accept finite int/float values, excluding bool and strings."""
12      if isinstance(value, bool) or not isinstance(value, (int, float)):
13          raise TypeError("Введите число, а не строку или логическое значение.")
14      try:
15          result = float(value)
16      except OverflowError as exc:
17          raise ValueError("Число слишком велико.") from exc
18      if not math.isfinite(result):
19          raise ValueError("Число должно быть конечным.")
20      return result
21
22
23  def _non_negative(value):
24      """Lengths and masses cannot be negative."""
25      result = _number(value)
26      if result < 0:
27          raise ValueError("Длина и масса не могут быть отрицательными.")
28      return result
29
30
31  def _finite_result(value):
32      """Reject an infinite result caused by floating-point overflow."""
33      if not math.isfinite(value):
34          raise ValueError("Результат слишком велик.")
35      return value
36
```

Приложение А Продолжение исходного кода

```
38  def celsius_to_fahrenheit(value):
39      """Degrees Celsius -> Fahrenheit; Celsius >= -273.15."""
40      value = _number(value)
41      if value < -273.15:
42          raise ValueError("Температура ниже абсолютного нуля.")
43      return _finite_result(value * (9 / 5) + 32)
44
45
46  def fahrenheit_to_celsius(value):
47      """Degrees Fahrenheit -> Celsius; Fahrenheit >= -459.67."""
48      value = _number(value)
49      if value < -459.67:
50          raise ValueError("Температура ниже абсолютного нуля.")
51      return _finite_result((value - 32) * (5 / 9))
52
53
54  def km_to_miles(value):
55      """Kilometres -> miles; kilometres >= 0."""
56      return _finite_result(_non_negative(value) * MILES_PER_KM)
57
58
59  def miles_to_km(value):
60      """Miles -> kilometres; miles >= 0."""
61      return _finite_result(_non_negative(value) / MILES_PER_KM)
62
63
64  def kg_to_pounds(value):
65      """Kilograms -> pounds; kilograms >= 0."""
66      return _finite_result(_non_negative(value) * POUNDS_PER_KG)
67
68
69  def pounds_to_kg(value):
70      """Pounds -> kilograms; pounds >= 0."""
71      return _finite_result(_non_negative(value) / POUNDS_PER_KG)
```

Исходный файл в комплекте: core.py. Версия с намеренной ошибкой: versions/v1_0/core.py. Интерфейс размещен отдельно в app.py и не включен в этот листинг вычислительного модуля.

Приложение Б Проверки и источники результатов

Показательные проверки чужого проекта TextStats. Полный набор находится в tests_initial и tests_improved; ниже приведены выдержки, сохраняющие смысл фактических проверок.

```
import pytest
from core import is_palindrome, count_words, word_frequency

def test_palindrome_lexicographic_regression():
    assert is_palindrome("яа") is False

@pytest.mark.parametrize("text, expected", [
    ("по-русски don't abc123def", 6),
    ("123_²", 0),
    ("e\u0301cole", 2),
])
def test_word_boundaries(text, expected):
    assert count_words(text) == expected

def test_frequency_unicode_casefold():
    assert word_frequency("Straße STRASSE") == {"strasse": 2}
```

Итоговые строки журналов pytest:

```
1 failed, 16 passed in 0.37s
17 passed in 0.31s
30 passed in 0.29s
```

Подтверждающие файлы в папке Белов_АМ_TextStats:

docs/specification.md

docs/bug_report.md

docs/mutation_analysis.md

verification/facts.json

verification/coverage_initial.json; coverage_improved.json

verification/mutations_initial.sqlite; mutations_improved.sqlite

Официальные источники статуса и области стандартов:

ГОСТ 19.401-78: https://protect.gost.ru/gost/details/e71e0951-ee1a-441f-b0c0-be3f3681c2f0

ГОСТ 34.602-2020: https://protect.gost.ru/gost/details/5876f733-ee91-431a-8cb6-9e02ea2436ac

Приложение В Проверка интерфейса

Введено 100, выбрано преобразование °C → °F, нажата клавиша Enter. Получен результат 212 °F. Это отдельная проверка запуска и связи интерфейса с вычислениями.

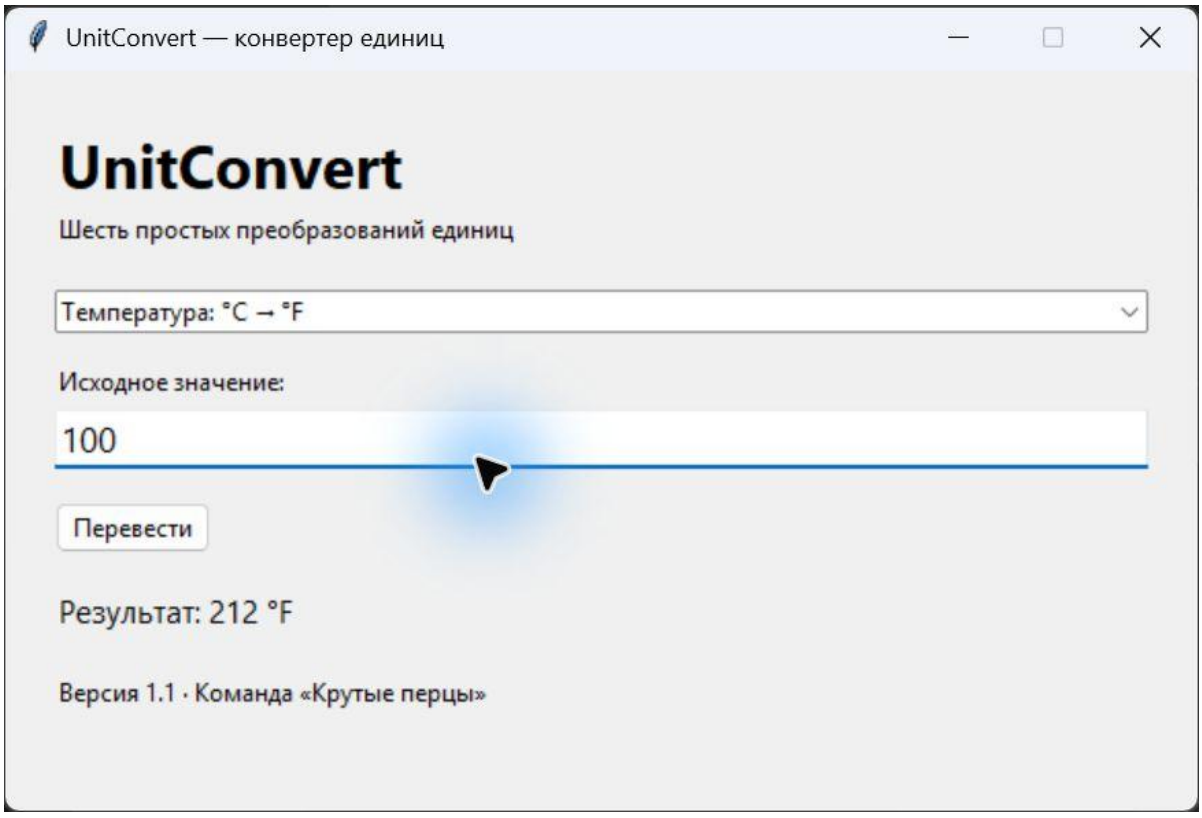


Рисунок В.1 — Фактический результат работы UnitConvert

Подтверждающие файлы в папке Молдаванский_РД_UnitConvert:

verification/screenshots/ui_conversion.png

verification/screenshots/ui_invalid.png